

# CITS3402/CITS5507 Assignment 1: Parallel Hash Collision Attack

[Your Name], 24914408

Semester 2, 2026

## Birthday-attack algorithm and collision-detection data structure

`toy_hash` produces a 48-bit output, so by the birthday bound a matching pair between two independent families of nonce trials is expected after roughly  $\sqrt{\pi/2 * 2^{48}} \sim 2.1e7$  combined trials — far fewer than the  $2^{48}$  a preimage search needs. Rather than fixing one file’s nonce and searching only the other, the program grows two pools of trials in parallel, one per file, stopping as soon as a trial’s hash already exists in the *other* file’s pool.

Both pools live in one shared open-addressing hash table keyed by the 48-bit hash. Each slot stores the hash, the nonce that produced it, which file it came from, and a “ready” flag. On insertion the hash is mixed with a Fibonacci multiplier and masked to the (power-of-two) table size to pick a home slot, then linear probing finds the first free or matching slot. A matching slot already owned by the *other* file is a genuine collision; one owned by the *same* file is a redundant self-collision, discarded. Capacity is four times the trial budget ( $4 * 2^{26}$  slots by default) so the load factor stays at or below 0.5 and probe chains stay short. Nonces are generated sequentially via an atomic counter per file rather than randomly — `toy_hash`’s MurmurHash3-style finalising mix already scatters sequential inputs uniformly, giving the same statistics as random sampling while wasting no nonce on a repeat.

## Parallelisation and synchronisation with OpenMP

Threads split evenly into an “A-group” and a “B-group” (even/odd thread id) inside one `#pragma omp parallel` region; each hashes trials only for its assigned file, drawing the next nonce with `atomic_fetch_add` on a per-file counter — no two threads ever hash the same nonce, and none ever blocks waiting for work.

The shared hash table is the only structure threads communicate through, made safe without any critical section or `omp_lock_t`. Each slot’s hash field is a C11 `_Atomic uint64_t` written only via `atomic_compare_exchange_strong` from `TABLE_EMPTY` to the trial’s hash; exactly one thread can ever win that CAS for a given slot, so that slot’s `nonce/owner` fields are subsequently written by that thread alone — no two threads ever write the same memory. The remaining hazard is a reader (another thread probing the same slot) observing it as claimed before the winner finishes writing `nonce/owner`; this is closed with a “published” flag — the winner writes `nonce/owner` first, then stores `ready = 1` with `memory_order_release`, and a thread that finds a matching hash spins on `ready` with `memory_order_acquire` before reading them, so it can never observe a torn write. That spin window is only the few instructions between the CAS succeeding and the stores after it, so it costs nothing in practice. A second atomic (`found_flag`) is claimed with one final CAS so only the first thread to discover a genuine collision records the winning nonce pair; every other thread

observes the shared `stop_flag` and exits. Before any file is written, the program independently recomputes `toy_hash` on the final bytes and aborts if they disagree, so a bug in the concurrent path can never silently produce an invalid submission.

## Memory requirements and trade-offs

The dominant cost is the collision table: at the default  $2^{26}$ -trials-per-side budget, capacity is  $2^{28}$  slots, each holding an 8-byte hash, 8-byte nonce, and two 1-byte tags — about 4.3 GB. This was chosen over a smaller, higher-load-factor table because linear probing degrades sharply as load factor approaches 1, and Kaya nodes have far more memory than this requires, so speed is favoured over economy. A second, much smaller cost is per-thread: each thread keeps one private, mutable copy of whichever file(s) it hashes (loaded once, header patched every trial), so only the 16-byte nonce is rewritten per trial rather than re-copying the whole file — for the largest (~900 KB) pair at 96 threads this is under 90 MB, negligible next to the table.

## Performance metrics and analysis

`search_seconds` measured on a 10-core development machine (every result independently verified against `check_toy_hash.py`) for the smallest pair confirms both correctness and near-linear early scaling:

| threads | search_seconds | speedup |
|---------|----------------|---------|
| 1       | 21.53          | 1.00x   |
| 2       | 10.69          | 2.02x   |
| 4       | 6.00           | 3.59x   |
| 8       | 3.78           | 5.69x   |
| 10      | 3.42           | 6.29x   |

Both runs found ~10.39M trials per side (~20.8M combined), matching the birthday-bound estimate almost exactly. Efficiency drops from 100% at 2 threads to ~63% at 10 threads: the search is memory-bandwidth-bound (every insert/probe is an effectively random access into a multi-gigabyte table), so once all cores are active they contend for cache/DRAM bandwidth rather than compute — a trend expected to continue at Kaya’s higher core counts. Solving full pairs end-to-end shows trial count roughly constant across file sizes while wall-clock time scales with them: `1_kilo` (64 KB) took 182.7 s (10.12M trials/side), `2_mega` (128 KB) took 314.5 s (8.96M trials/side) — 1.7x slower for 2x the size, since bigger files cost more per hash but need a similar trial count. This is why the harder pairs need Kaya’s full 96-core node: `3_giga-6_exa` extrapolate to tens of minutes at 10 threads, only fitting the 900 s budget once ~96 cores are used.

*[Insert Kaya results here: `search_seconds` for all six pairs across thread counts 1–96 from `scripts/scaling_job.slurm`, with the resulting speedup/efficiency curves.]*